Embedded Control for Robotics

Lecture # 09 Control Design

Book: Embedded Digital Control with Microcontrollers
Chapter # 8

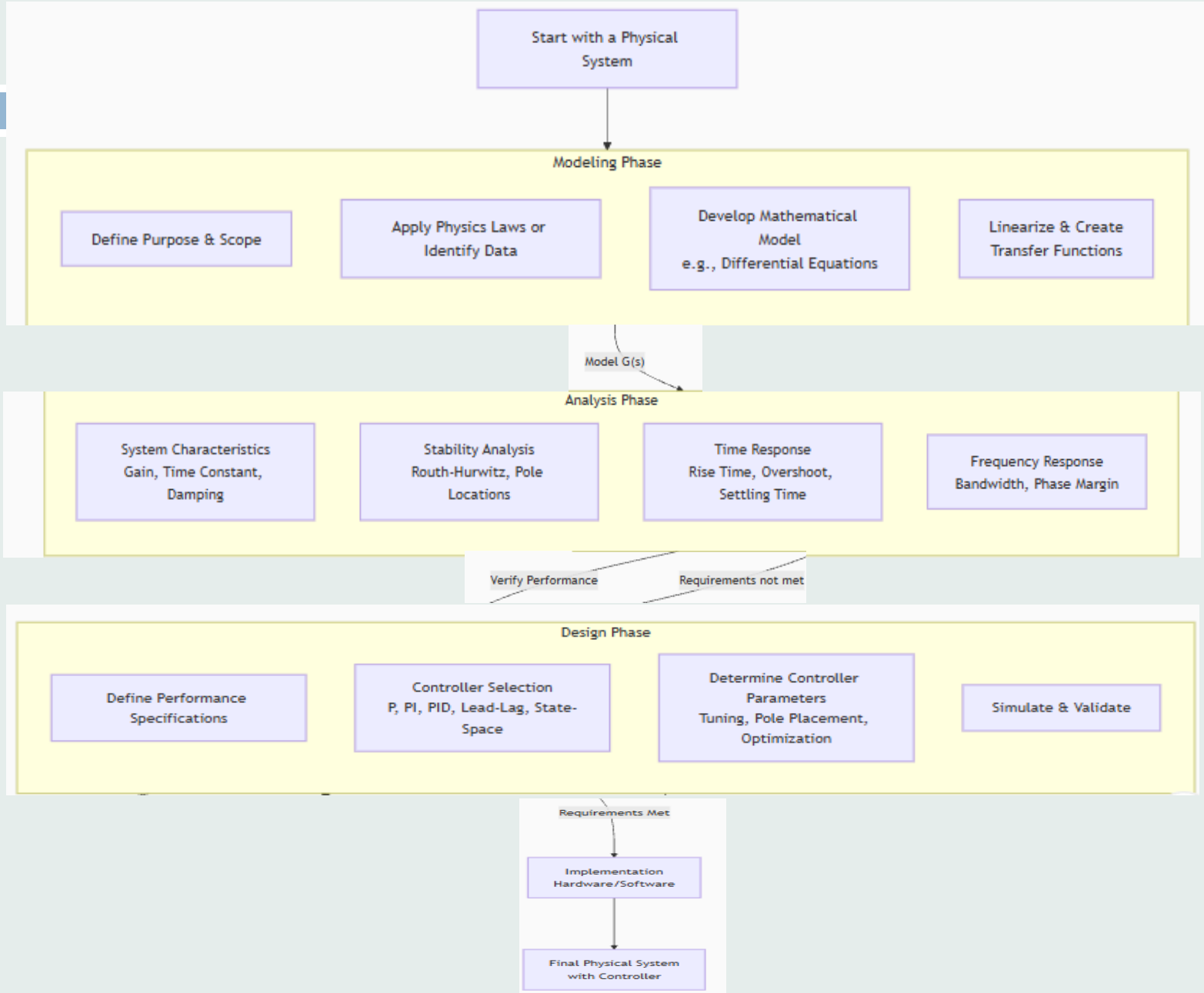
Dr. Suleman Khan
CS, NUCES-FAST, Islamabad

Overview Today's Lecture

- Control System
- On-off Control
- PID Controller
- PID Tuning
- Control Design given Specification

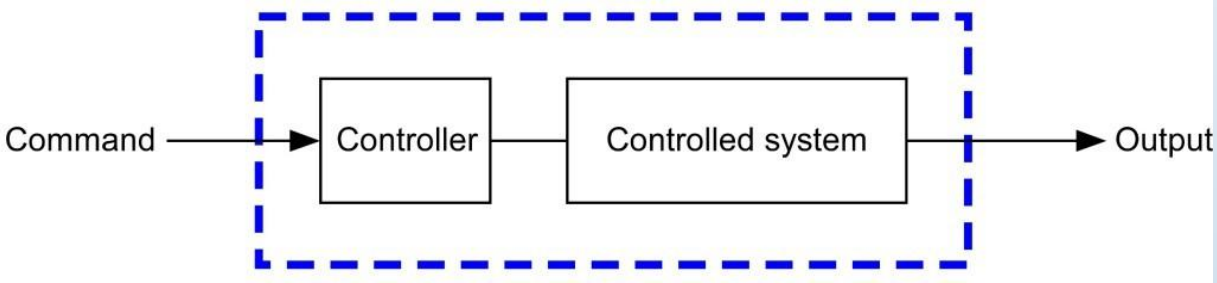
Control Systems: Big Picture

1. Modeling
2. Analysis
3. Design



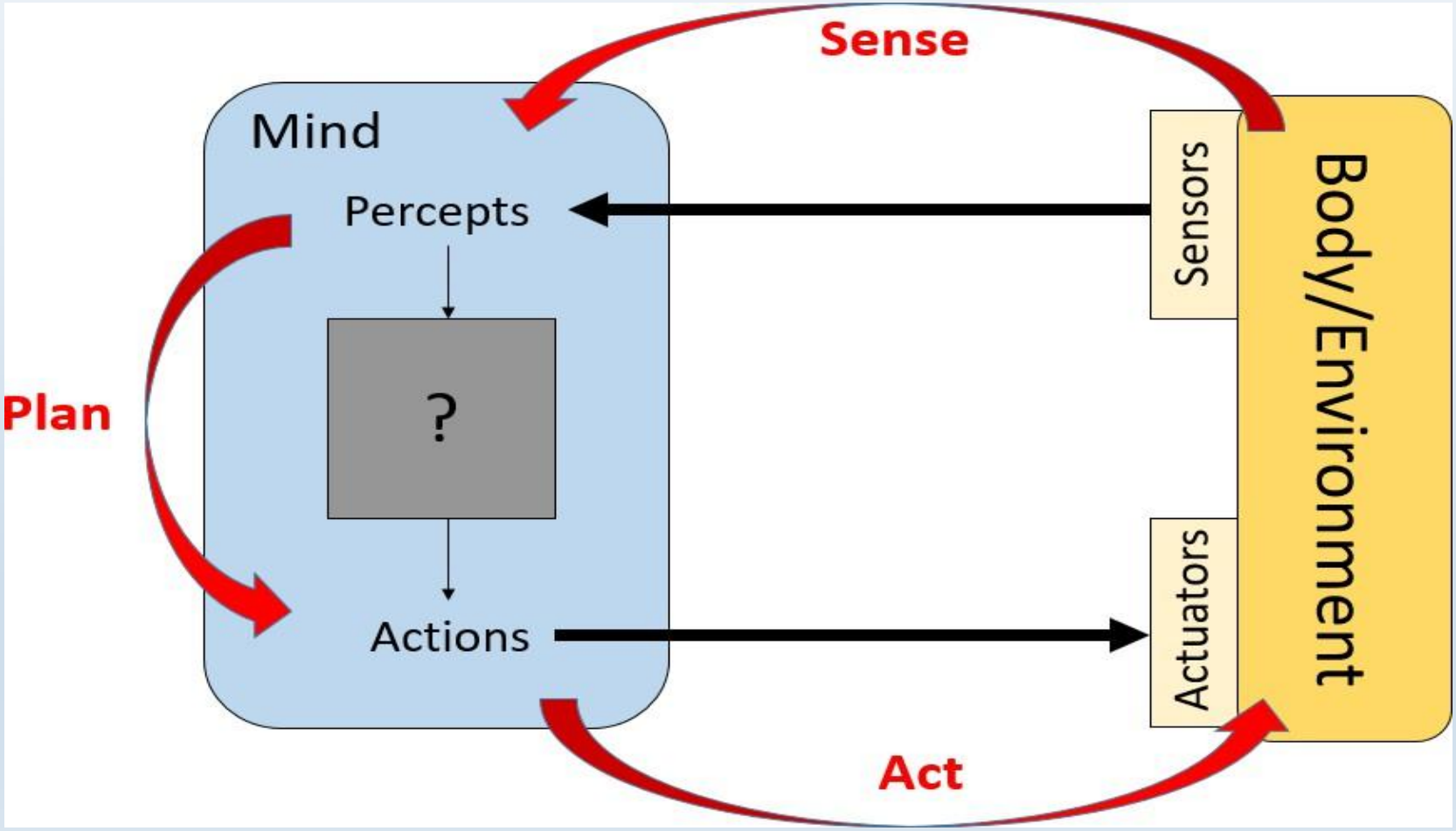
Control System

- A control system should contain a controller to perform various control functions on the system to get the desired outputs.



- A complete control system should be composed of inputs, the controller, the system or a process, and the system outputs.

Big Picture


Sense
Mind
Percepts
?
Plan
Actions
Sensors
Actuators
Body/Environment
Act

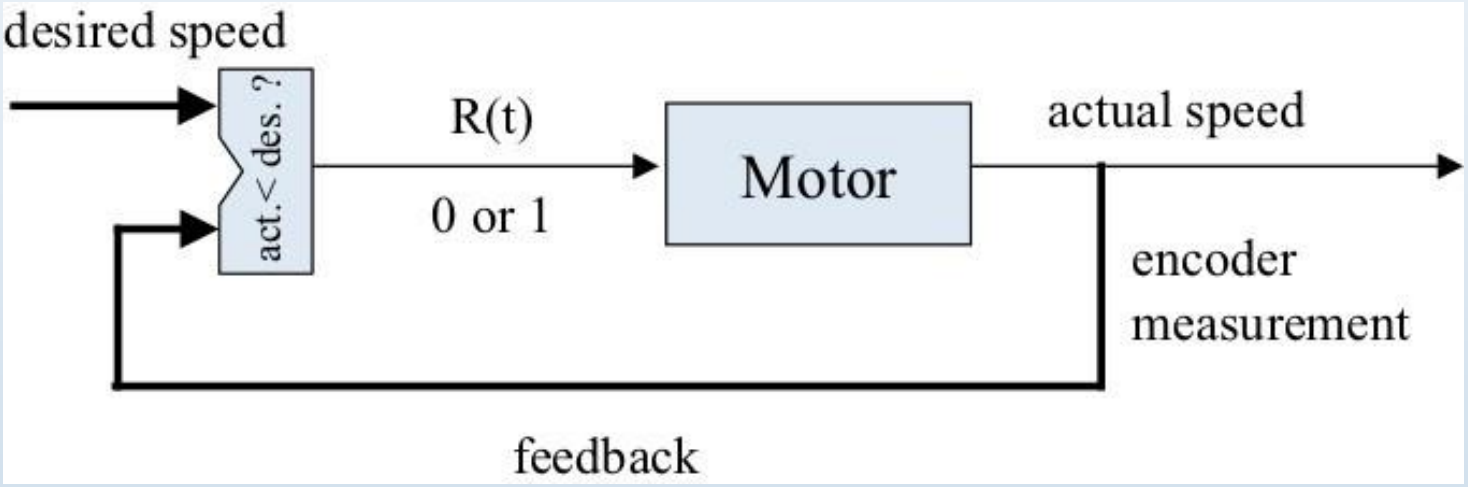
On-Off Control

- **If** actual speed is **lower** than desired speed:
  $\rightarrow$ *Increase motor power.*
- **If** actual speed is **higher** than desired speed:
  $\rightarrow$ *Decrease motor power.*

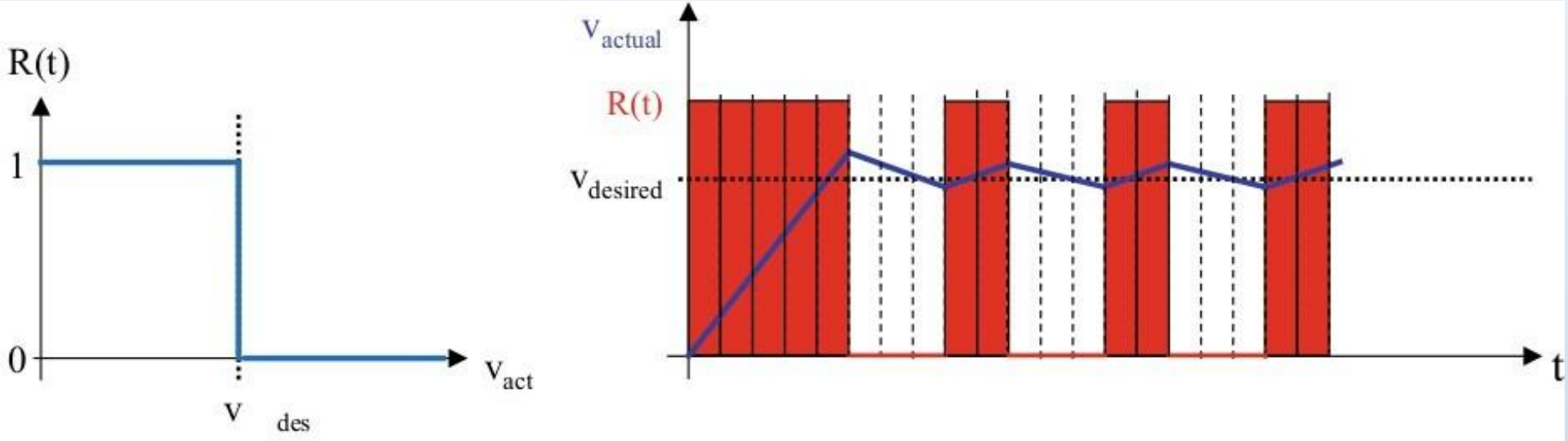
$R(t)$	motor output function over time t
$v_{act}(t)$	actual measured motor speed at time t
$v_{des}(t)$	desired motor speed at time t

$$R(t) = \begin{cases} 1 & \text{if } v_{act}(t) < v_{des}(t) \\ 0 & \text{else} \end{cases}$$

On-Off Control



On-Off Control: System Behavior



On-Off Control:

- Simple to Implement
- Cost-effective
- Quick response
- Used by:
  - refrigerator,
  - heater,
  - thermostat,
  - light dimmer

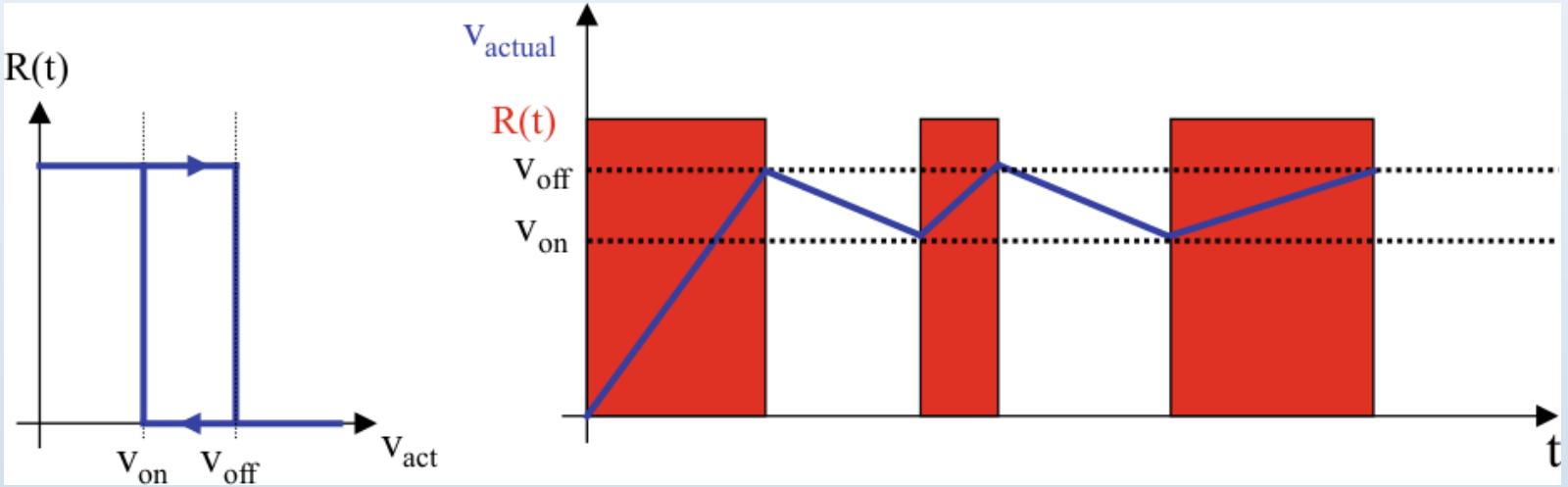
On-Off Control with Hysteresis

- hysteresis band, which consists of two desired values, one for switching on and one for switching off.
- This prevents a high switching frequency near the desired value, in order to avoid excessive wear

On-Off Control with Hysteresis

$$R(t+\Delta t) = \begin{cases} 1 & \text{if } v_{act}(t) < v_{on}(t) \\ 0 & \text{if } v_{act}(t) > v_{off}(t) \\ R(t) & \text{else} \end{cases}$$

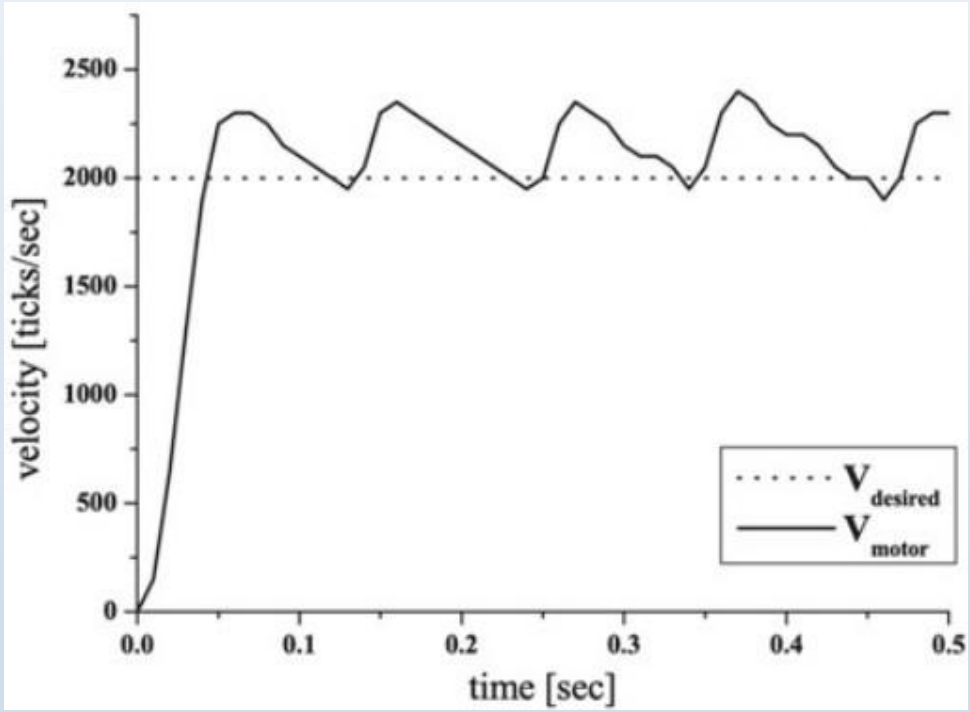
On-Off Control with Hysteresis



On-Off Control Implementation

1. We need a subroutine for calculating the motor signal as defined by the formula.
   This subroutine has to:
   a. Read encoder data (*input*)
   b. Compute new output value R(t)
   c. Set motor speed (*output*)
2. The subroutine has to be called periodically (for example, every 1/100 s).
   Since motor control is a *low-level* task, we would like this to run in the background and not interfere with any user program.

On-Off Control Step response



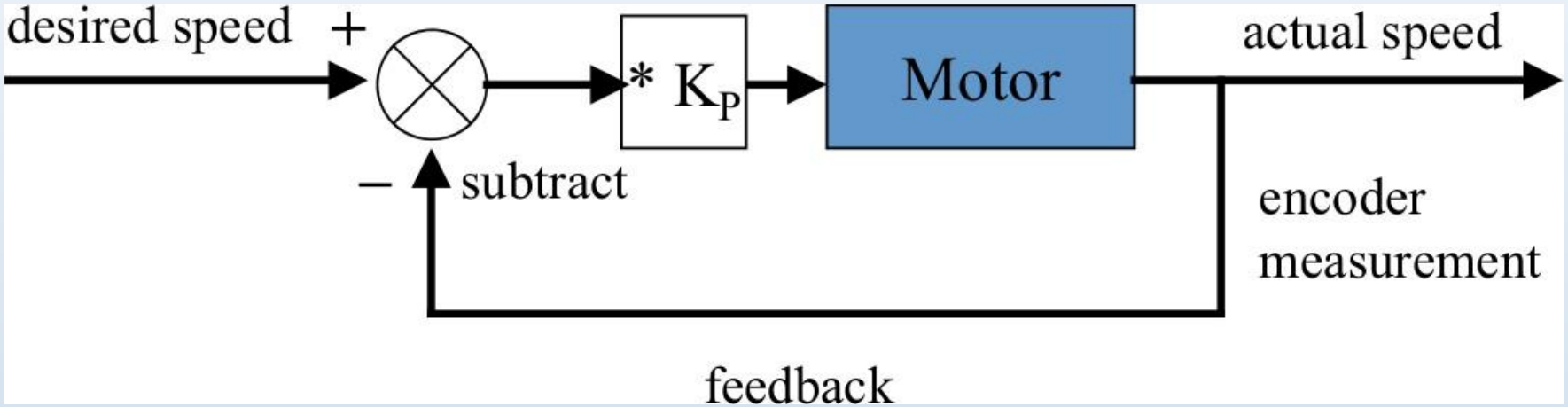
On-Off Control Problem

Abrupt motor control change between fully on and off does not result in steady state smooth control of the syytem

PID Controller

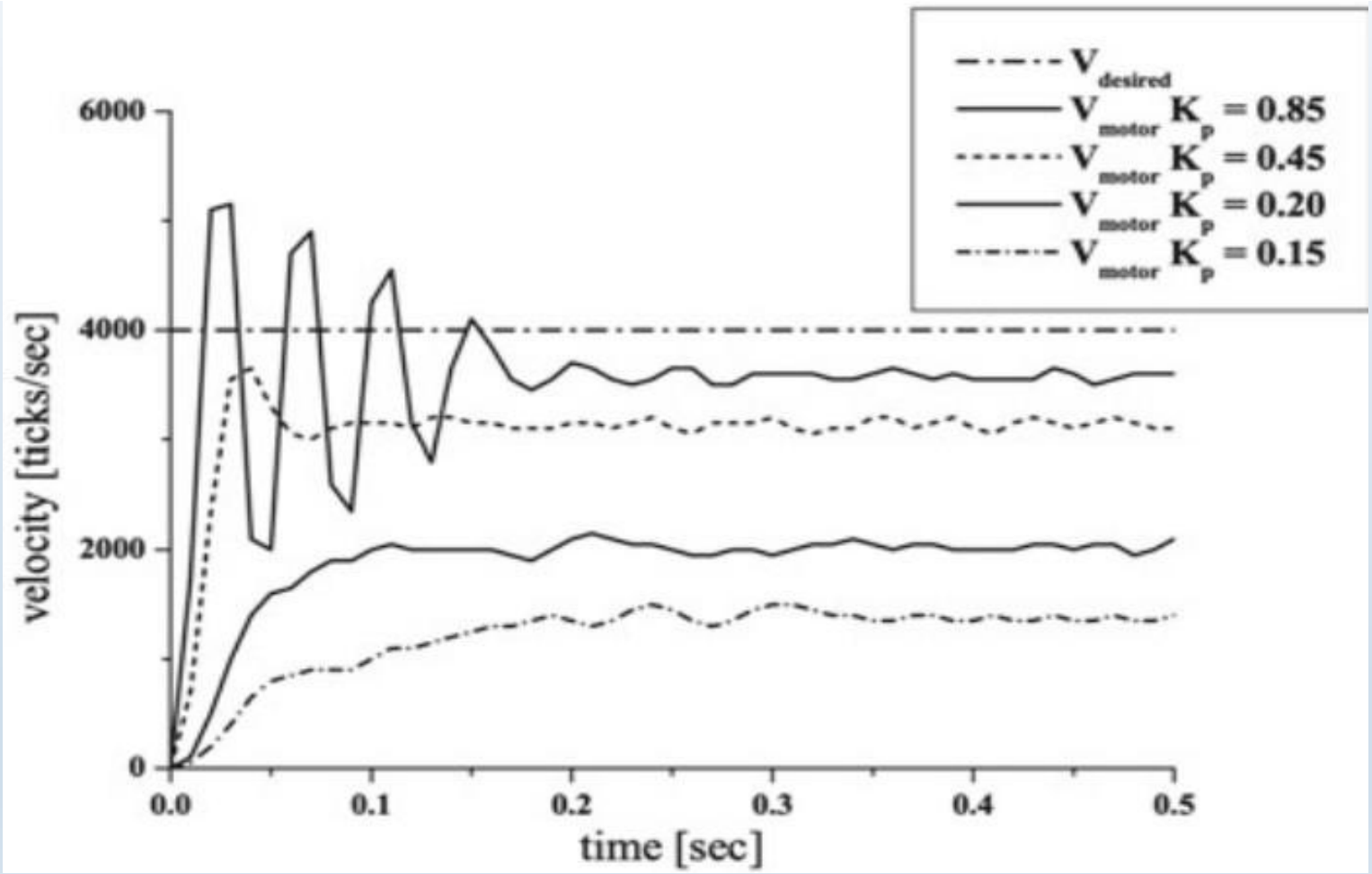
- Simplest method of control is not always the best.
- An advanced controller is PID
- Industry standard
- It comprises a proportional, an integral and a derivative control part
- PID = P + I + D

Proportional Controller



$$R(t) = K_P \cdot (v_{des}(t) - v_{act}(t))$$

Proportional Response



Proportional Controller Response

- P controller's equilibrium state is not at the desired velocity.
- If the desired speed is reached exactly, the motor output is reduced to zero, as defined in the P controller formula shown above. Therefore, each P controller will keep a certain steady-state error from the desired velocity, depending on the controller gain KP.
- Higher the gain KP, the lower the steady-state error

Limitation of Proportional controller

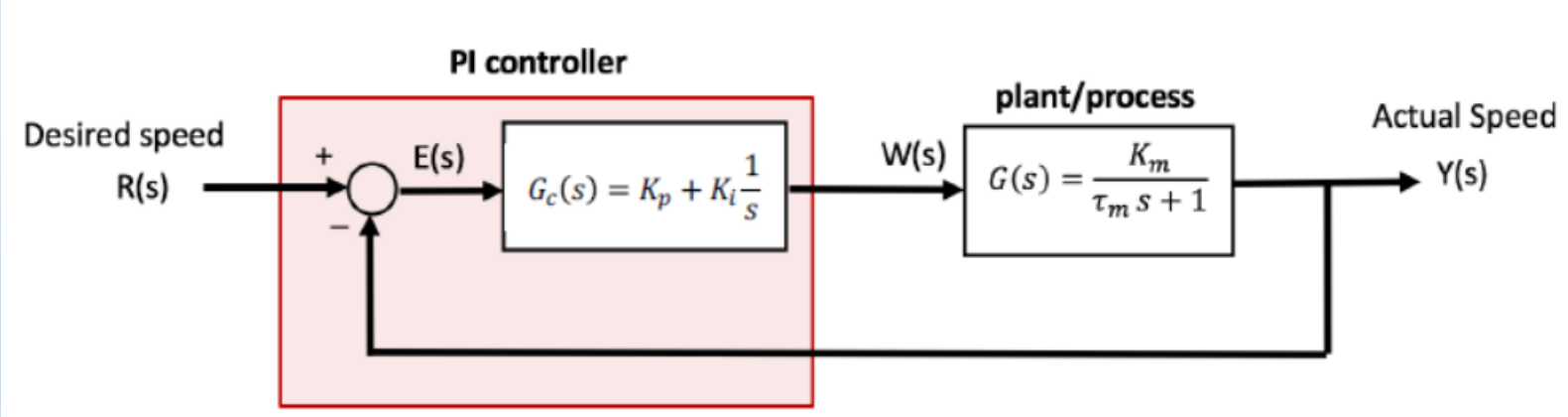
- Using P-only control is simple, but often insufficient because:
- 1. If Kp is small, error e(t) can be large (i.e., there is an offset error between the set-point and controlled output variable, in this case, the speed of the motor).
- 2. If Kp is large, the system may oscillate (i.e., become unstable).
- 3. Even if the system is stable, it may take a long time to settle to its final output value or exhibit large overshoots.
- 4. It may not have sufficient tolerance to perturbations or disturbances

Proportional Integral(PI) Controller

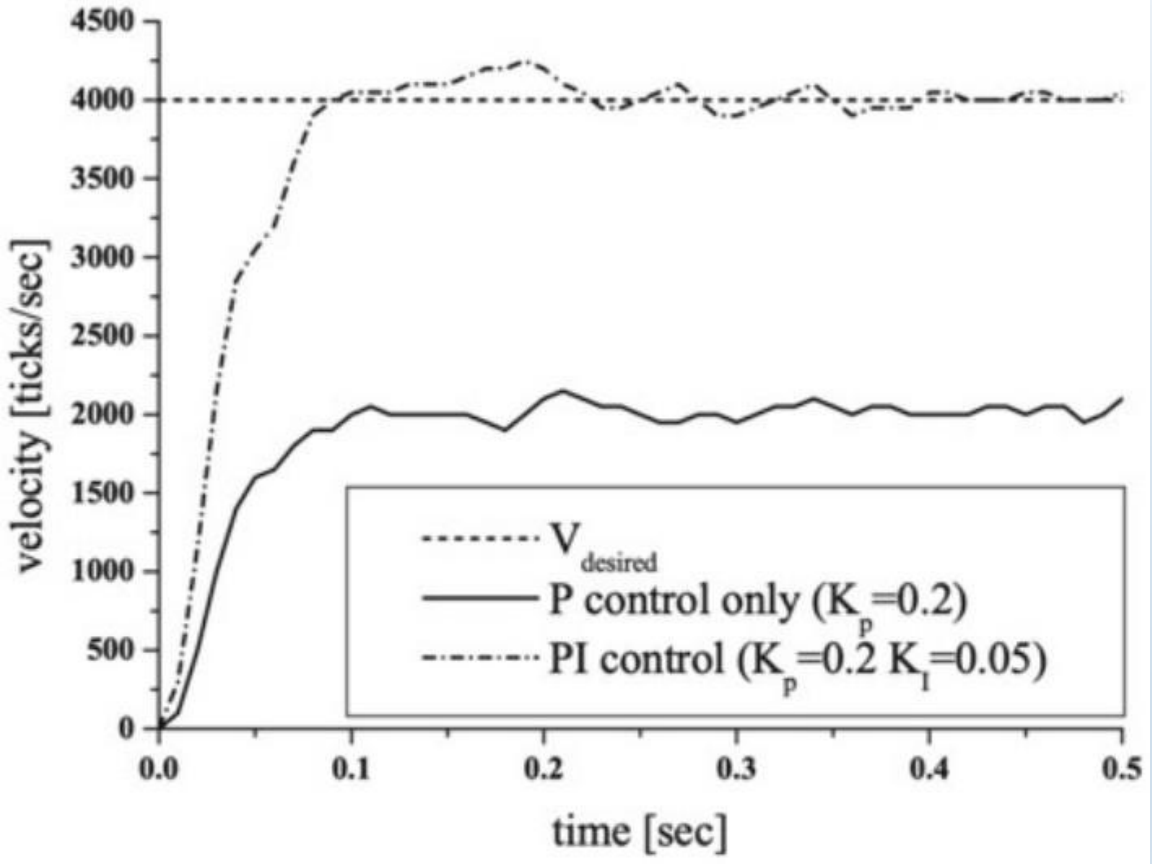
- Unlike the P controller, the I controller (integral controller) is rarely used alone, but mostly in combination with the P or PD controller.
- Idea for the I controller is to reduce the steady-state error of the P controller.
- With an additional integral term, this steady- state error can be reduced to zero
- I term also help in stability.

PI Controller

$$R(t) = K_P \cdot e(t) + Q_I \cdot \int_0^t e(t)dt$$



PI Controller Response



PI Controller

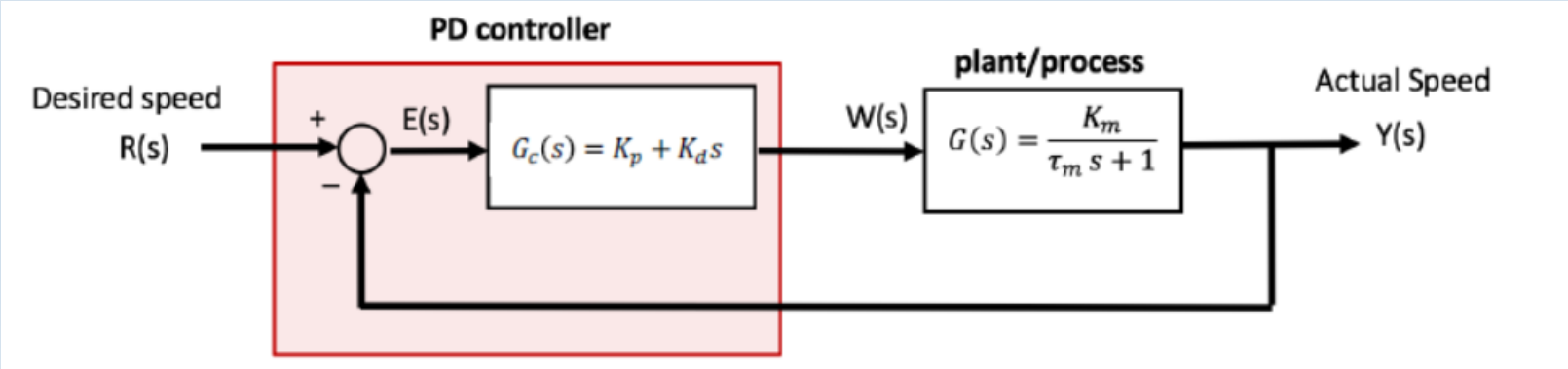
The proper way of implementing a PI controller starts with discretization, replacing the integral with a sum, using the trapezoidal rule:

$$R_n = K_P \cdot e_n + Q_I \cdot t_{delta} \cdot \sum_{i=1}^{n} \frac{e_i + e_{i-1}}{2}$$

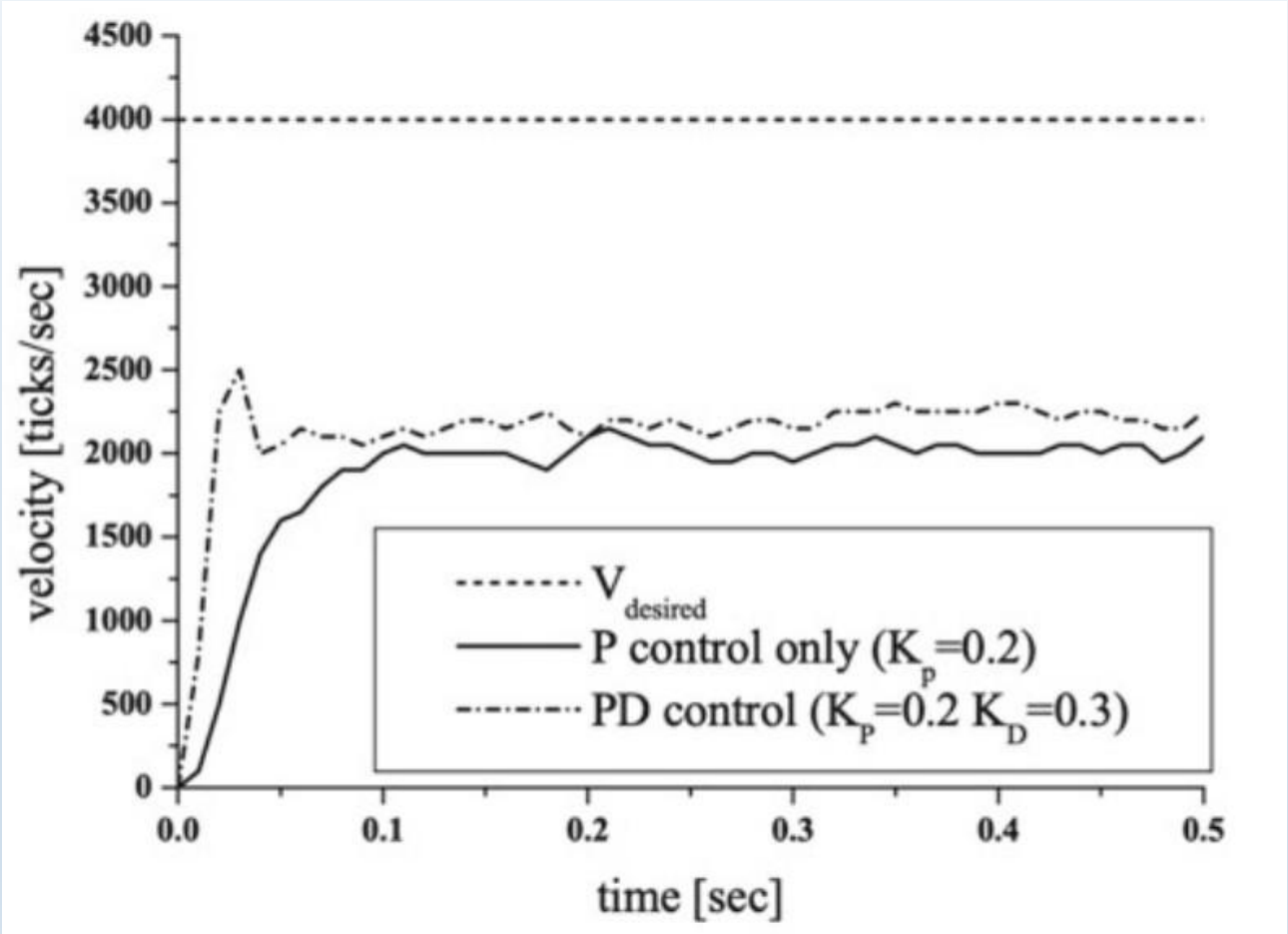
$$R_n = R_{n-1} + K_P \cdot (e_n - e_{n-1}) + K_I \cdot (e_n + e_{n-1})/2$$

Derivative Controller

- Similar to the I controller, the D controller (derivative controller) is rarely used by itself, but mostly in combination with the P or PI controller.
- The idea for adding a derivative term is to speed up the P controller's response to a change of input.
- The PD controller reaches equilibrium faster than the P controller, but still has a steady- state error.



P vs PD


4500
4000
3500
3000
2500
2000
1500
1000
500
0
velocity [ticks/sec]
0.0
0.1
0.2
0.3
0.4
0.5
time [sec]
V desired
P control only (K p =0.2)
PD control (K P =0.2 K D =0.3)

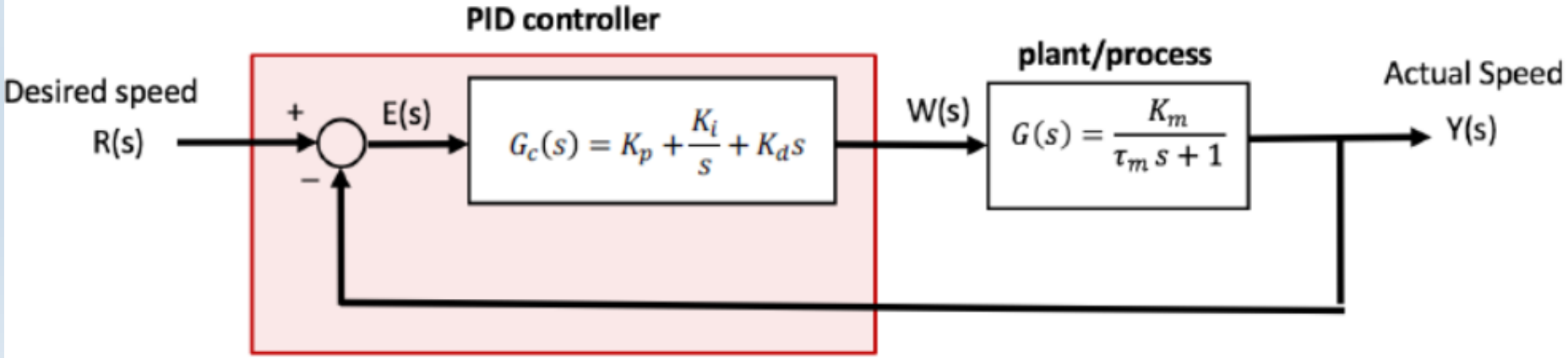
PID Controller

- Finally, we can combine all three terms to form a PID controller:

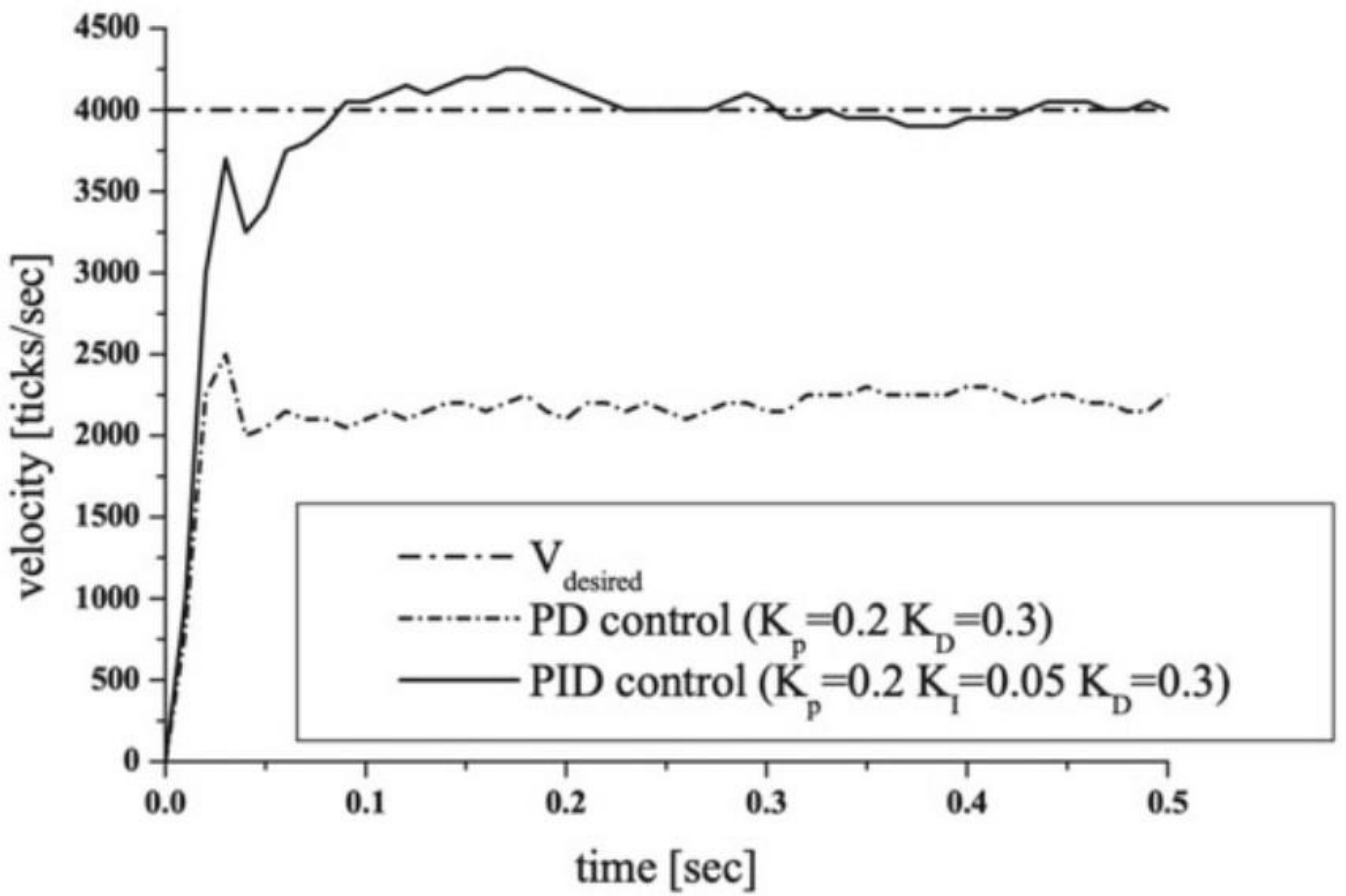
$$u(t) = K_p e(t) + K_i \int e(\tau) d\tau + K_d \dot{e}(t)$$

- This has the advantages of **ALL** three types of feedback control (P, I and D):

1. **Removal** of steady-state **error** due to I.
2. **Reduce** the amount of **overshoots** (due to be I and D).
3. **Improve** the **transient** response to make it faster (due to both I and D).
4. **Improve stability** of the system.
5. Better perturbation tolerance.



P vs PID


4500
4000
3500
3000
2500
2000
1500
1000
500
0
velocity [ticks/sec]
0.0
0.1
0.2
0.3
0.4
0.5
time [sec]
V desired
PD control (K p =0.2 K D =0.3)
PID control (K p =0.2 K I =0.05 K D =0.3)

PID Summary

- P: makes the controller responsive
- I: eliminates steady state error
- D: resists the change and removes oscillations

- So should P, I and D be higher or lower?
- How to select values?

PID Demo

- https://chev.me/pid-demo/

Effect of Adding a Pole/Zero

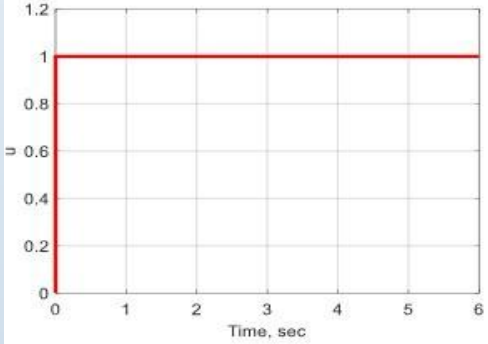
- LHP Zeros: Increase overshoot, decrease rise time, and do not affect settling time.
  - The effects are small if the zero is far in the LHP.
- RHP Zeros: Cause undershoot but no effect on settling time.
  - The effects are small if the zero is far in the RHP.
- LHP Poles: Increase settling time.
  - The effects are small if the pole is far in the LHP.
- RHP Poles: ?

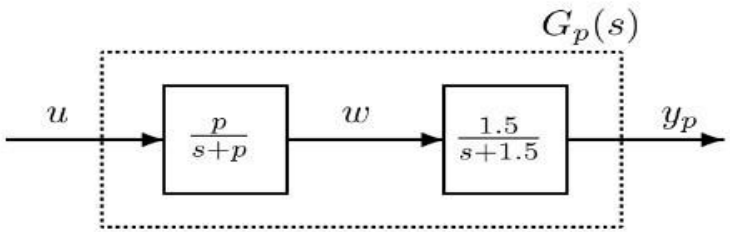
Effect of Adding a Pole

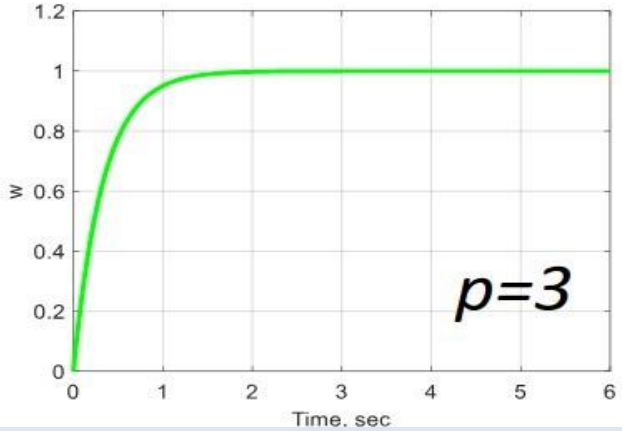
Consider the second-order system:

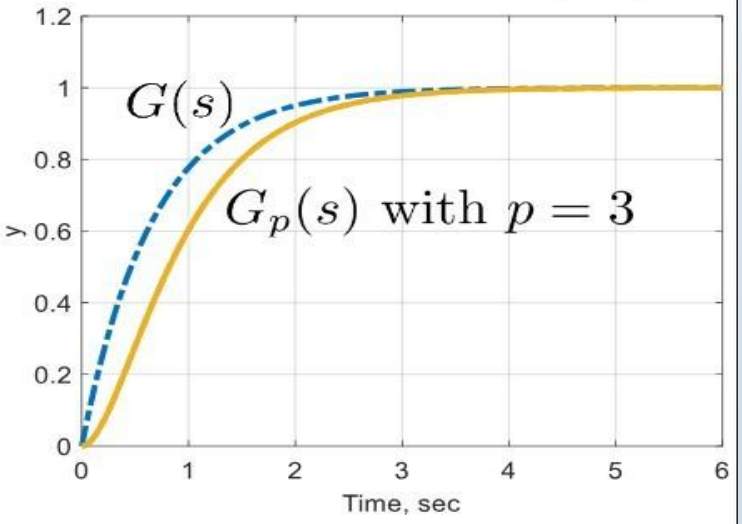
$$\ddot{y}(t) + (1.5 + p)\,\dot{y}(t) + (1.5p)\,y(t) = (1.5p)\,u(t)$$

$$G_p(s) = \frac{1.5p}{s^2+(1.5+p)s+1.5p} = \frac{p}{s+p} \cdot G(s) \text{ where } G(s) = \frac{1.5}{s+1.5}$$









Effect of Adding a LHP Pole

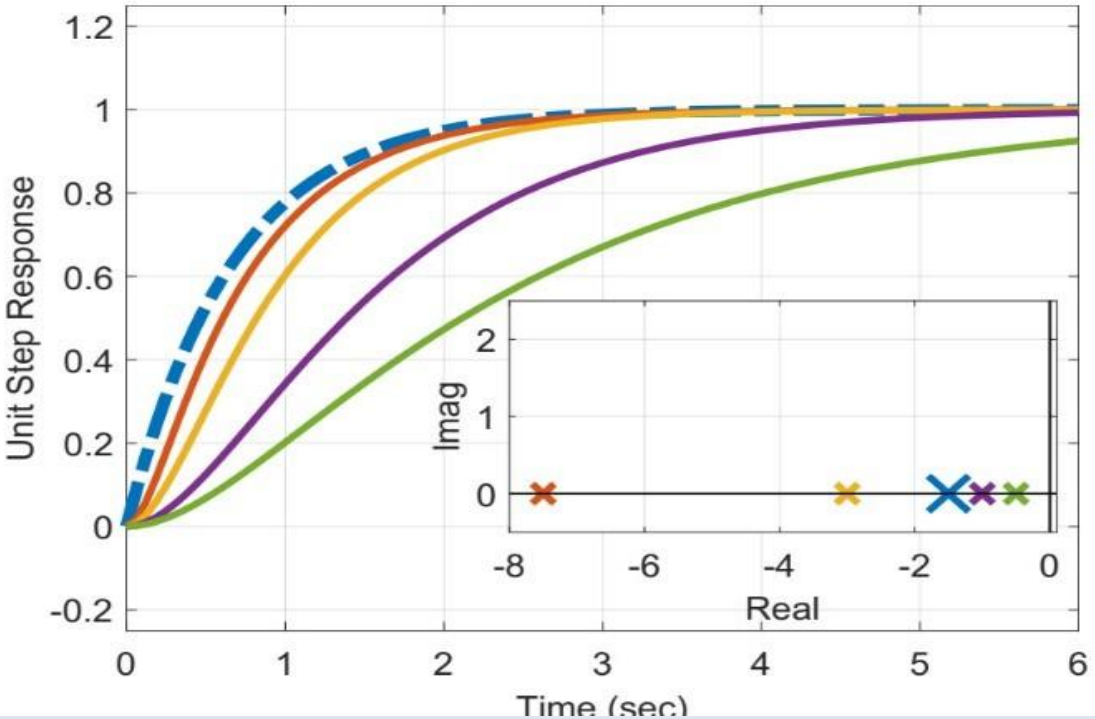
Consider the second-order system:

$$\ddot{y}(t) + (1.5 + p)\,\dot{y}(t) + (1.5p)\,y(t) = (1.5p)\,u(t)$$

$$G_p(s) = \frac{1.5p}{s^2+(1.5+p)s+1.5p} = \frac{p}{s+p} \cdot G(s) \text{ where } G(s) = \frac{1.5}{s+1.5}$$

Additional poles in the LHP increase the settling time.

The effect is small if the extra pole is far in the LHP ($\approx 5\times$faster than slowest pole)

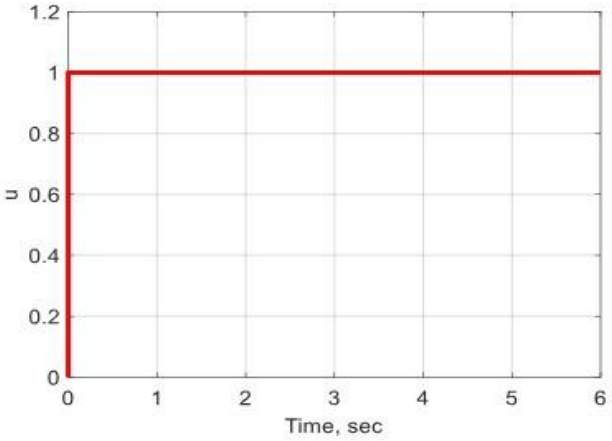


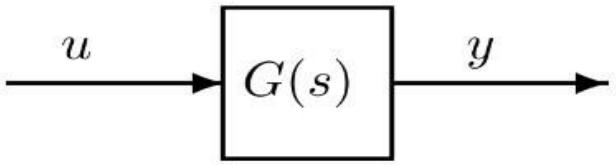
Effect of Adding a Zero

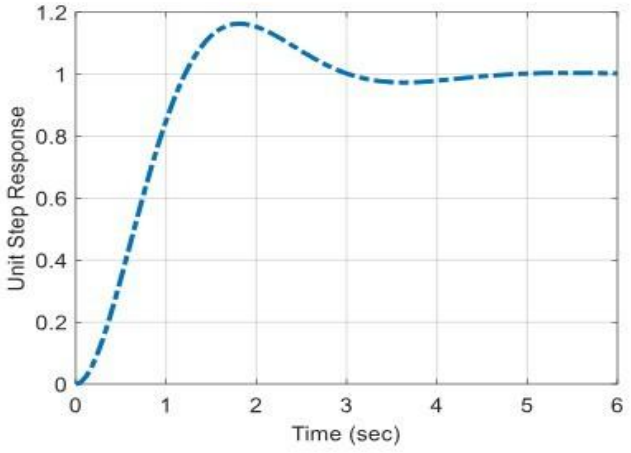
Consider the second-order system:

$$\ddot{y}(t) + 2\,\dot{y}(t) + 4\,y(t) = (4b)\,\dot{u}(t) + 4u(t)$$

$$G_b(s) = \frac{4b\,s+4}{s^2+2s+4} = (bs+1)\cdot G(s) \text{ where } G(s) = \frac{4}{s^2+2s+4}$$







Effect of Adding a LHP Zero

Consider the second-order system:

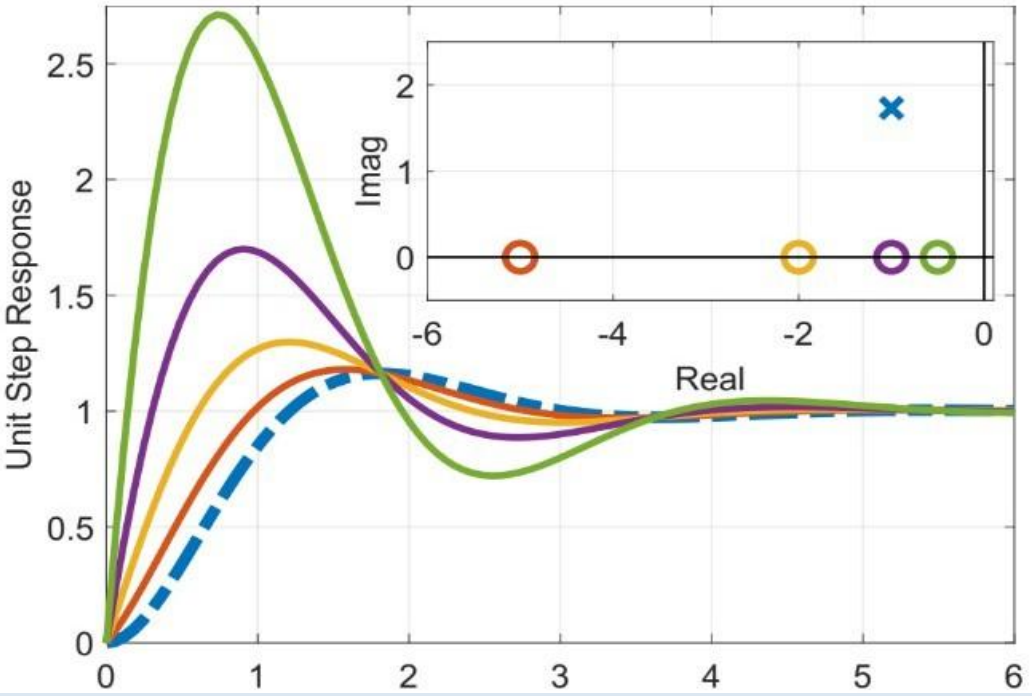
$$\ddot{y}(t) + 2\,\dot{y}(t) + 4\,y(t) = (4b)\,\dot{u}(t) + 4u(t)$$

$$G_b(s) = \frac{4b\,s+4}{s^2+2s+4} = (bs+1)\cdot G(s) \text{ where } G(s) = \frac{4}{s^2+2s+4}$$

A zero in the LHP:

- **Increases overshoot**
- **Decreases rise time**
- **No effect on settling time**

The effects are small if the zero is far in the LHP.



Effect of Adding a RHP Zero

Consider the second-order system:

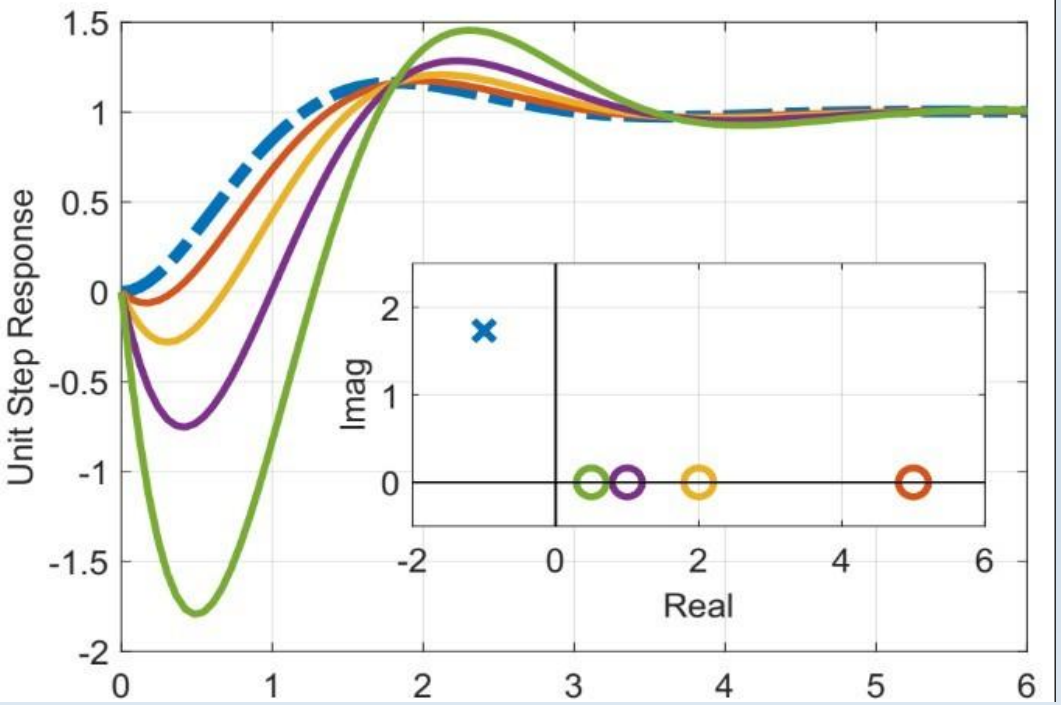
$$\ddot{y}(t) + 2\,\dot{y}(t) + 4\,y(t) = (4b)\,\dot{u}(t) + 4u(t)$$

$$G_b(s) = \frac{4b\,s+4}{s^2+2s+4} = (bs+1)\cdot G(s) \text{ where } G(s) = \frac{4}{s^2+2s+4}$$

A zero in the RHP:

- **Causes undershoot**
- **No effect on settling time**

The effects are small if the zero is far in the RHP.



Effect of Adding a Pole: Root Locus

- Repulsion of poles: Poles repel other poles on the root locus, influencing their movement away from the added pole as the gain K increases.
- Slower system response: Adding a pole often slows down the system, increasing the rise time and settling time.
- Increased oscillations: Additional poles can decrease damping, leading to more oscillations or instability in the transient response.
- Reduced stability margin: Poles may shift closer to the right half-plane, reducing the system's stability margin, especially for higher gains.
- More complex root locus paths: The root locus paths become more intricate, potentially introducing more complex conjugate pole pairs that oscillate.
- Potential for destabilization: If the added pole is near the imaginary axis or in the right half- plane, the system may become unstable at certain gain values.

Effect of Adding a Zero: Root Locus

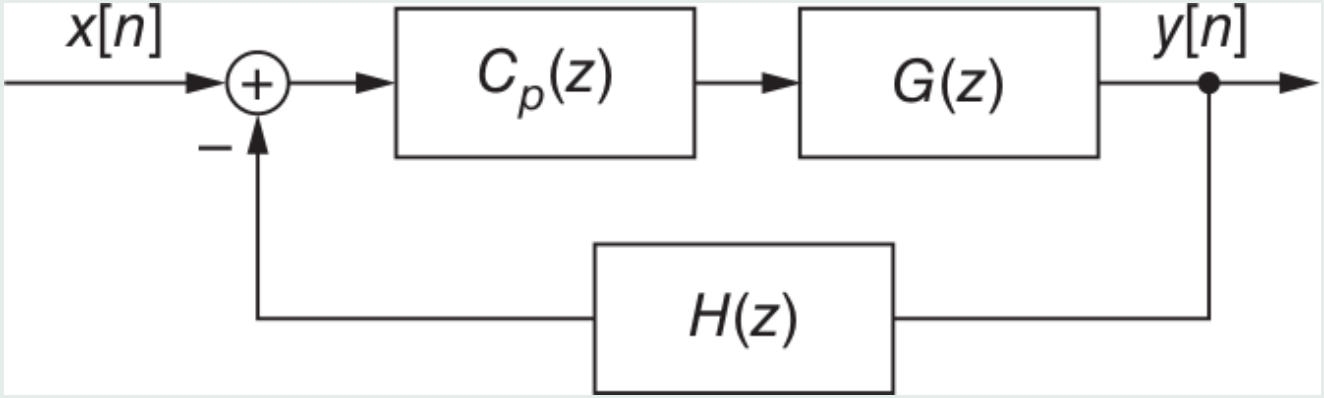
- Attraction of poles: Zeros act as attractors on the root locus. As the gain KK increases, poles tend to move toward the zeros.
- Modified pole paths: The paths that poles follow on the root locus change. The closer a zero is to the poles, the more it influences the trajectory.
- Faster system response: Adding a zero can speed up the system response, often reducing rise time.
- Increased overshoot: Zeros can cause more overshoot in the transient response, especially if they are located near the imaginary axis.
- Improved damping: Properly placed zeros can increase damping, reducing oscillations in the system response.
- Possible destabilization: Depending on the location of the zero, it can pull poles toward the right half-plane, potentially making the system unstable for high gain values.

Effect of Adding a Pole/Zero

- Demo notebook

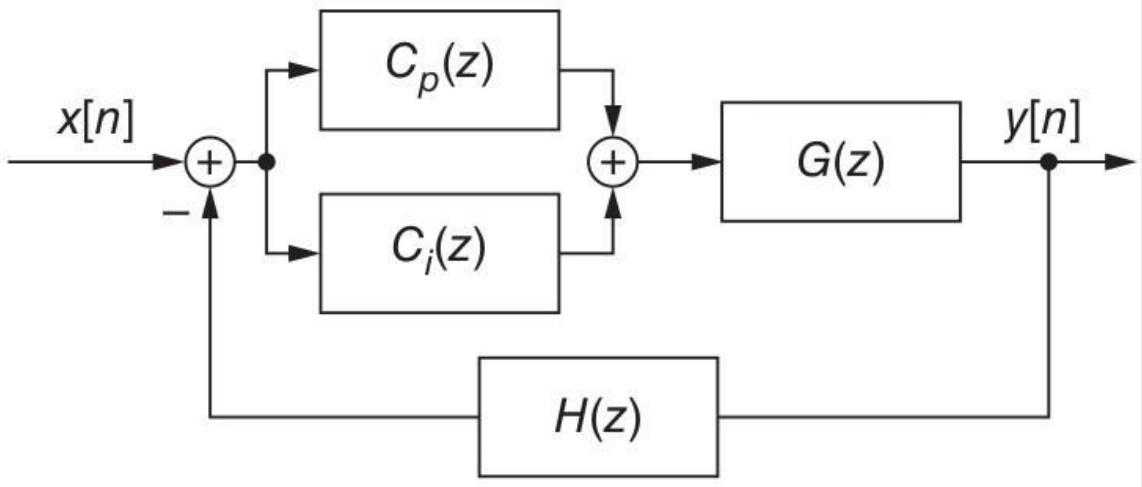
P Structure

- Cp (s) = Kp



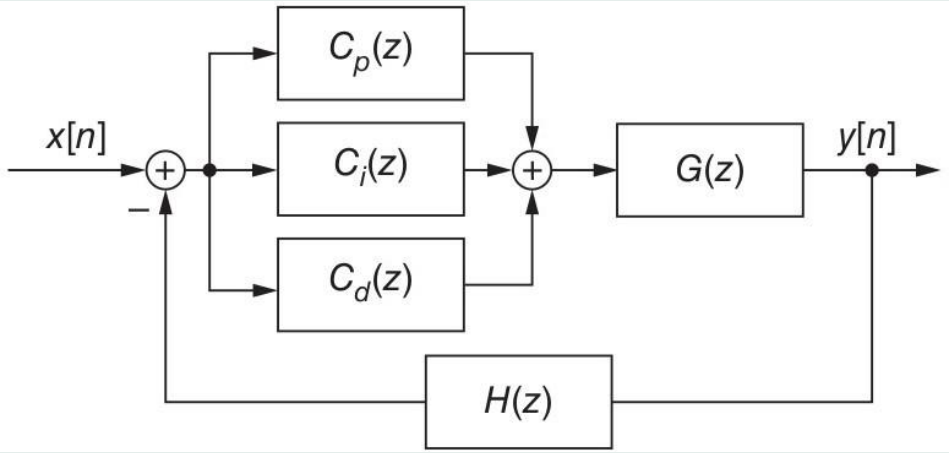
PI Structure

- Cp(s) = Kp + Ki/s
- Cp(s) = (sKp + Ki)/s
- Cp(s) = (s + Ki/Kd)/s



PID Structure

- $Cp(s) = Kp + Ki/s + sKd$
- $Cp(s) = (s^2Kd+sKp + Ki)/s$



PID Summary

- PID controller adds Poles and Zeros
- These Poles/Zeros influence the system's transient and steady-state response
- Exact location of Poles/Zeros depends upon Kp, Ki and Kd gains which needs to be tuned

Tuning PID

- Choosing the correct values for K_p, K_d and K_i is known as **tuning** the controller.
- **Impact of** various **gains** on step response of a system:

PID Gain	Percent Overshoot	Settling Time	Steady-State Error
Increasing K_P	Increases	Minimal impact	Decreases
Increasing K_I	Increases	Increases	Zero steady-state error
Increasing K_D	Decreases	Decreases	No impact

- Manual Method
- The Ziegler–Nichols Method
- The Cohen–Coon Method
- The Chien–Hrones–Reswick Method

Manual method of tuning PID

1. Set K_p , K_d and K_i to zero.
2. Start with a small K_p, double it each time until the system starts to oscillate.
3. Half the value of K_p.
4. Start with a small K_d, double it each time until the system starts to oscillate.
5. Half the value of K_d .
6. Start with a small K_i, double it each time until the system starts to oscillate.
7. Half the value of K_i.

- If you are only using PD or PI controller, skip the irrelevant steps.
- Fine tune the various gain until you get the response you want.

Ziegler-Nichols method of tuning the PID

Step: 1

Three auxiliary parameters obtained from the response of the system to be controlled are

1. *K = System Gain*

$$K = \frac{y(\infty)-y(0)}{x(\infty)-x(0)}$$

2. *L* = is the system dead-time = $t1 - \tau$

t1 = time to reach 63.2% of the step response

τ = 1.5 * time to reach 28.3 % of step response

3. a

$$a = K\frac{L}{\tau}.$$

$$G(s) = Ke^{-Ls}/(\tau s + 1).$$

Continue…

Controller	K_p	K_i	K_d
P	$1.0/a$	—	—
PI	$0.9/a$	$K_p/(3L)$	—
PID	$1.2/a$	$K_p/(2L)$	$K_pL/2$

The Cohen–Coon Method

Apart from a and L parameter Uses R = L / tau

Table 8.2 Auxiliary parameters for the Cohen–Coon method.

Controller	T_i	T_d
PI	$L((30 + 3R)/(9 + 20R))$	—
PID	$L((32 + 6R)/(13 + 8R))$	$4L/(11 + 2R)$

Table 8.3 PID gain parameters according to the Cohen–Coon method.

Controller	K_p	K_i	K_d
P	$(1/a)(1 + (R/3))$	—	—
PI	$(1/a)((9/10) + (R/12))$	K_p/T_i	—
PID	$(1/a)((4/3) + (R/4))$	K_p/T_i	K_pT_d

The Chien–Hrones–Reswick Method

Table 8.4 Auxiliary parameters for the Chien–Hrones–Reswick method.

Controller	T_i	T_d
PI	$2.3L$	—
PID	$2L$	$0.42L$

Table 8.5 PID gain parameters according to the Chien–Hrones–Reswick method.

Controller	K_p	K_i	K_d
P	$0.7/a$	—	—
PI	$0.7/a$	K_p/T_i	—
PID	$1.2/a$	K_p/T_i	K_pT_d

PID Tuning Python


```
[Kp, Ki, Kd] =
    mpcontrolPC.pid_tune(gs, 'ZieglerNichols', 'P')

[Kp, Ki, Kd] =
    mpcontrolPC.pid_tune(gs, 'ZieglerNichols', 'PI')

[Kp, Ki, Kd] =
    mpcontrolPC.pid_tune(gs, 'ZieglerNichols', 'PID')
```

P controller

Listing 8.1: Obtaining the P controller for the DC motor.

```
import mpcontrolPC

#Continous-time System
num = [165702.47]
den = [1, 390.2, 15701]

gs = mpcontrolPC.tf(num, den)

[Kp, Ki, Kd] = mpcontrolPC.pid_tune(gs, 'ZieglerNichols', 'P')

#Analog P Controller
num2 = [Kp]
den2 = [1]
cs = mpcontrolPC.tf(num2, den2)

#Digital P Controller
Ts = 0.0005
cz = mpcontrolPC.c2d(cs, Ts)
```

Step response of Closed Loop


```
import mpcontrol
import mpcontrolPC

#System
Ts = 0.0005
num1 = [0.00943, 0.01886, 0.00943]
den1 = [1, -1.8188, 0.8224]

#Transfer function of the system
g = mpcontrolPC.tf(num1,den1,Ts)
print(g)

#Transfer function of the controller, Kp
num2 = [0.729409]
den2 = [1]

c = mpcontrolPC.tf(num2,den2,Ts)

#Transfer function of the feedback system
num3 = [1]
den3 = [1]

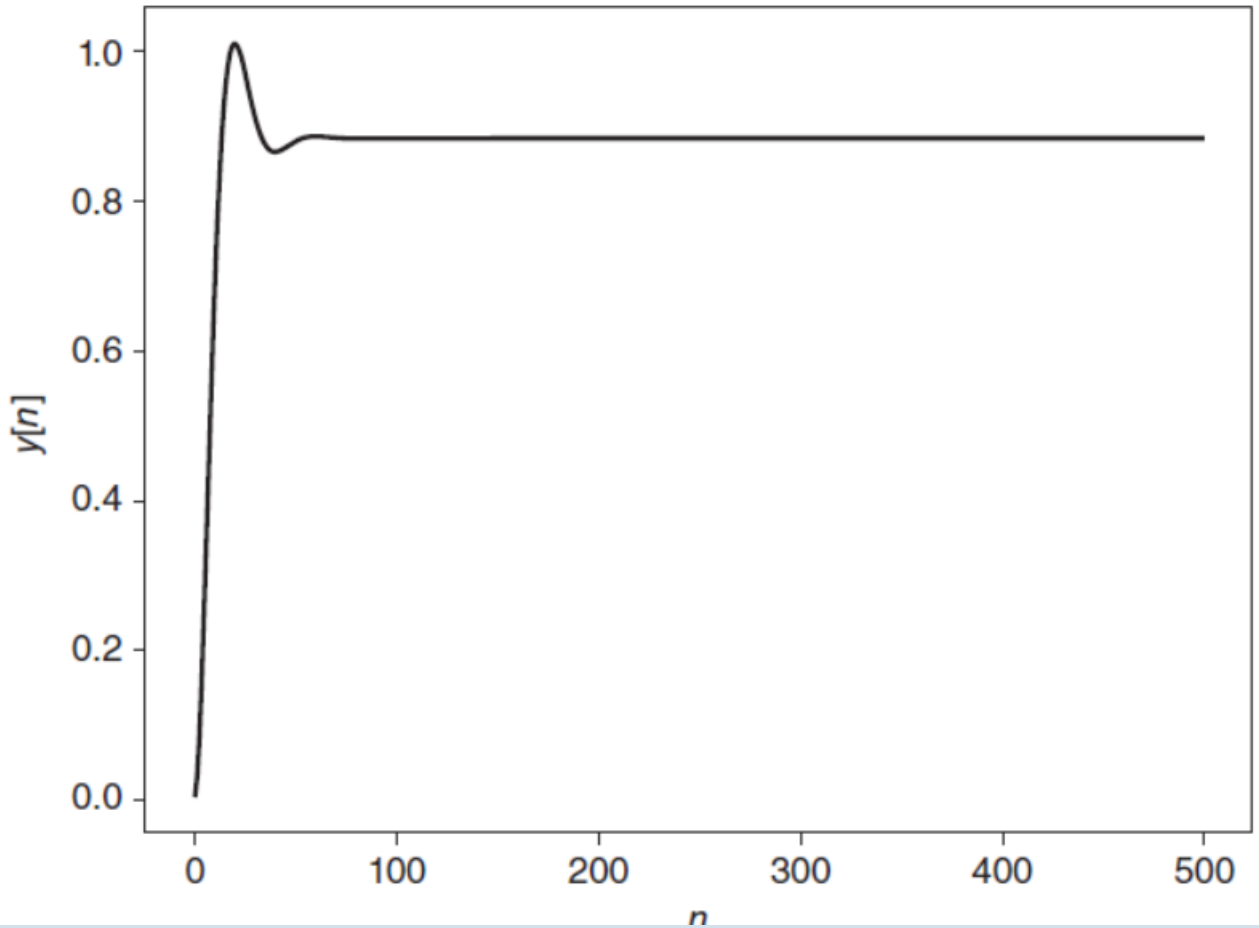
h = mpcontrolPC.tf(num3,den3,Ts)

# Transfer function of the closed-loop controlled system
gol = mpcontrolPC.series(g,c)
gcl = mpcontrolPC.feedback(gol,h)

#Step input signal
N = 500
x = mpcontrol.step_signal(N, 1)

#Calculate the output signal
y = mpcontrol.lsim(gcl, x)
```

Step Output of the close loop




```
gcl.damp()

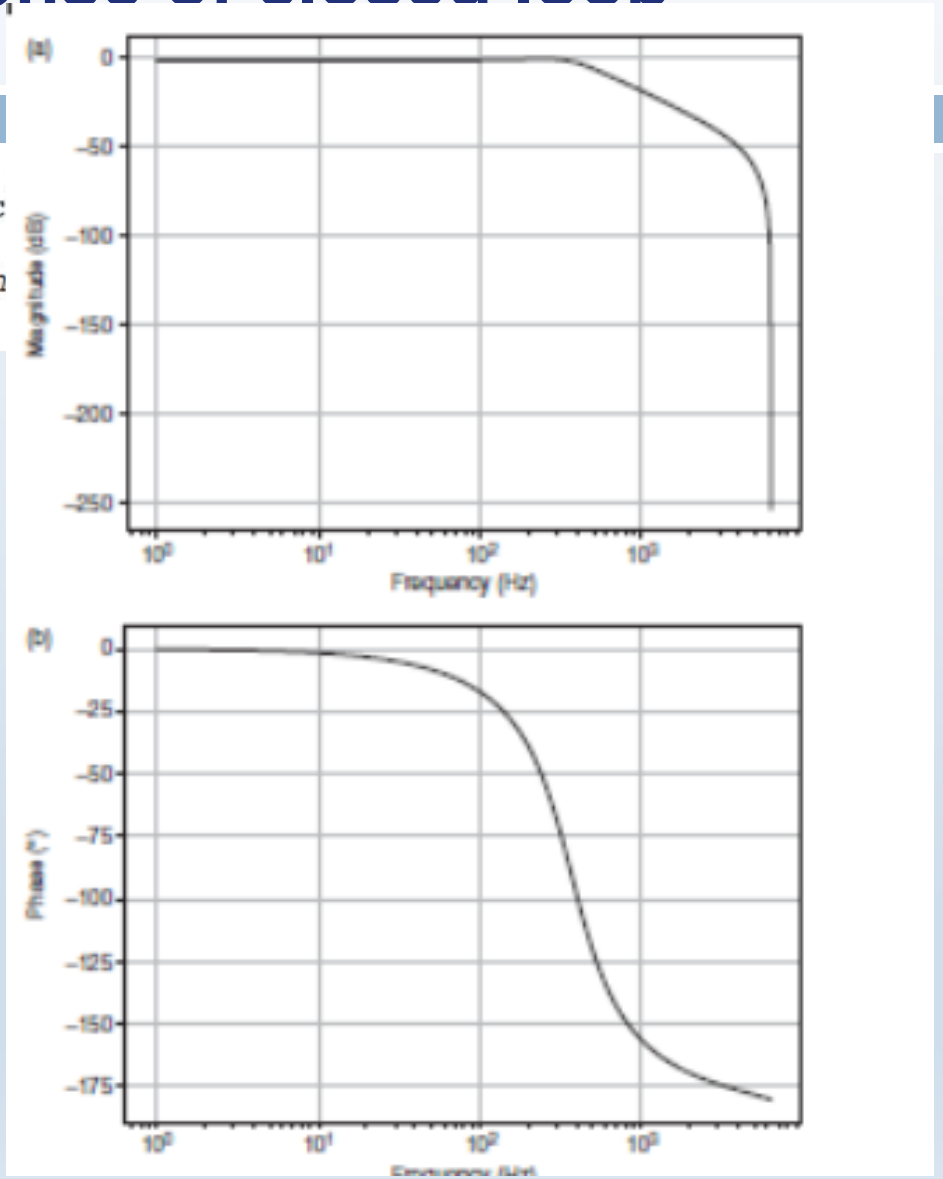
gcl.stepinfo()

ss_error = gcl.ss_error('Step')
```

Frequency response of closed loop


```
#Bode plot of the system
[mag, phase, freq] =  mpcontrolPC.bode(gc

#GM, PM, Crossover frequency values of th
mpcontrolPC.margin(gcl)
```



PID Controller

As the Python code in Listing 8.6 is executed, we obtain PID controller gain parameters as $K_p = 0.8753$, $K_i = 152.0259$, and $K_d = 0.0013$. Based on these values and bilinear transformation, the PID controller in digital form becomes

$$C(z) = \frac{5.9528z^2 - 10.0030z + 4.2022}{z^2 - 1} \tag{8.6}$$

PID Controller time Response

Table 8.6 Comparison of the P, PI, and PID controllers by the time domain performance criteria.

System	T_{rt} (sec)	M_p (%)	T_{pt} (sec)	T_{st} (sec)	e_{ss}
No controller	0.0485	0	0.4345	0.0880	−9.4778
With P controller	0.0045	14.2537	0.0100	0.0205	0.1157
With PI controller	0.0045	35.4436	0.0105	0.0280	−3.8968e−13
With PID controller	0.0040	27.0234	0.0095	0.0285	−4.0902e−14

PID Controller: Frequency Response

Table 8.7 Comparison of the P, PI, and PID controllers by the frequency domain performance criteria.

System	GM (dB)	PM (°)	ω_{cg} (rad/sec)	ω_{cp} (rad/sec)
No controller	313.071	53.094	6283.190	338.802
With P controller	313.071	∞	6283.190	Unavailable
With PI controller	378.426	64.781	6283.190	373.848
With PID controller	∞	105.343	Unavailable	352.644

PID Tuning Python

- Demo

Design controllers for robotic systems for stability and performance enhancement

Problem Setup

- Plant:

$$G(s) = \frac{1}{s(s+2)}$$

- Unity feedback system.
- Controller:

$$C(s) = K_p + K_d s$$

- Requirements:
  1. **Overshoot < 4.5%**
  2. **2% Settling time < 2.5 s**

Step 1: Relating Overshoot to Damping Ratio

For a 2nd-order system, overshoot is:

$$M_p = e^{-\frac{\zeta\pi}{\sqrt{1-\zeta^2}}}$$

Requirement: $M_p < 0.045$.

Solve for ζ:

$$0.045 > e^{-\frac{\zeta\pi}{\sqrt{1-\zeta^2}}}$$

Taking ln:

$$\ln(0.045) = -\frac{\zeta\pi}{\sqrt{1-\zeta^2}}$$

$$-3.101 = -\frac{\zeta\pi}{\sqrt{1-\zeta^2}}$$

$$\frac{\zeta}{\sqrt{1-\zeta^2}} = \frac{3.101}{\pi} \approx 0.987$$

$$\zeta \approx 0.70$$

✅ So damping ratio should be at least **0.7**.

Step 2: Settling Time Requirement

Settling time (2%) approximation:

$$T_s \approx \frac{4}{\zeta \omega_n}$$

Given $T_s < 2.5$:

$$\frac{4}{0.7\omega_n} < 2.5$$

$$\omega_n > \frac{4}{0.7 \cdot 2.5} \approx 2.29\,\text{rad/s}$$

✅ Natural frequency should be > **2.3 rad/s.**

Step 3: Desired Pole Location

Poles should be near:

$$s = -\zeta\omega_n \pm j\omega_n\sqrt{1-\zeta^2}$$

Choose slightly higher ω_n (say **3 rad/s**) for margin.

$$s = -0.7(3) \pm j\,3\sqrt{1-0.7^2}$$

$$s = -2.1 \pm j2.14$$

✅ Desired dominant poles: **-2.1 ± j2.14**

Step 4: Characteristic Equation with PD Controller

Closed-loop transfer function:

$$T(s) = \frac{C(s)G(s)}{1 + C(s)G(s)}$$

$$C(s)G(s) = \frac{K_p + K_d s}{s(s+2)}$$

Characteristic equation:

$$1 + \frac{K_p + K_d s}{s(s+2)} = 0$$

$$s(s+2) + K_d s + K_p = 0$$

$$s^2 + (2 + K_d)s + K_p = 0$$

But! That's a **second-order form** — however our plant is 2nd-order, adding PD keeps it 2nd-order. Perfect.

Step 5: Match Desired Characteristic Equation

Desired polynomial:

$$s^2 + 2\zeta\omega_n s + \omega_n^2$$

Substitute $\zeta = 0.7, \omega_n = 3$:

$$s^2 + 4.2s + 9$$

Now match coefficients:

$$2 + K_d = 4.2 \quad \Rightarrow \quad K_d = 2.2$$

$$K_p = 9$$

✅ Controller:

$$C(s) = 9 + 2.2s$$

Step 6: Verify Performance

- **Overshoot**: For $\zeta = 0.7$, overshoot $\approx 4.6\%$. Slightly above. If we pick $\zeta = 0.72$, it drops $<4.5\%$.
- **Settling time**: $T_s \approx \frac{4}{0.7 \cdot 3} = 1.9s < 2.5s$. ✅

So we can slightly increase damping ratio (by adjusting Kd a bit higher).

Final PD Controller

$$C(s) = 9 + 2.5s$$

(This ensures $\zeta \geq 0.72$ and $M_p < 4.5$).

Block diagram

Step 8: Root Locus / Pole Plot

- Without controller: poles at 0, -2 → unstable step response.
- With controller: dominant poles shift to -2.16 ± j2.0 (approx), satisfying both specs.

✅ **Answer Summary**

The designed PD controller is:

$$\boxed{C(s) = 9 + 2.5s}$$

It ensures <4.5% overshoot and settling time <2.5s.

Phyton

- Demo